

Neurónové siete (ANN)

Okruhy 2+3+4 · Poznámky ku skúške · UMI

Umelý neurón – matematický model a výpočet

- Biologická inšpirácia
 - Umelý neurón modeluje správanie biologického neurónu.
 - Prijíma vstupy, váhuje ich, sčíta a pošle na výstup cez aktivačnú funkciu.
- Formálny model (McCulloch–Pitts)
 - Vstupy: $x_1, x_2, \dots, x_n$
 - Váhy: $w_1, w_2, \dots, w_n$ (naučené parametry)
 - Bias: b – posun prahu, naučený parameter
 - Vážená suma: $z = \sum_i w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$
 - Výstup: $a = f(z)$, kde f je aktivačná funkcia
- Úloha váh a biasu
 - Váha w_i určuje, ako silno daný vstup ovplyvňuje výstup.
 - Bias b umožňuje posunúť aktivačnú funkciu – neurón sa aktivuje aj bez vstupu.
 - Bez biasu by neurón musel prechádzať počiatkom, čo obmedzuje vyjadriteľnosť.
- Výstup neurónu
 - Pre skrytú vrstvu: typicky ReLU, tanh...
 - Pre výstupnú vrstvu: závisí od úlohy (sigmoid, softmax, lineárna)
- Vrstvové zapojenie
 - Vstupná vrstva $\rightarrow 1..N$ skrytých vrstiev $\rightarrow$ výstupná vrstva.
 - Každý neurón vo vrstve l je prepojený s každým neurónom vrstvy $l-1$ (fully connected).

Aktivačné funkcie

- Prečo aktivačná funkcia?
 - Bez nej by sieť bola len lineárna transformácia, bez ohľadu na počet vrstiev.
 - Nelinearity umožňujú sieťam aproximovať ľubovoľnú funkciu.
- Sigmoid (logistická funkcia)
 - $\sigma(z) = \frac{1}{1 + e^{-z}}$, výstup $\in (0, 1)$
 - Unipolárna – kladné hodnoty
 - Vhodná pre binárnu klasifikáciu (výstupná vrstva)
 - Nevýhody: problém miznúceho gradientu (vanishing gradient) pri hlbokých sieťach, výstup nie je centrovný na nule.
- Tanh (hyperbolický tangens)

- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, výstup $\in (-1, 1)$
- Bipolárna, symetrická – centrovná na nule $\rightarrow$ lepšia ako sigmoid pre skryté vrstvy
- Stále trpí vanishing gradient v extrémnych hodnotách.
- ReLU (Rectified Linear Unit)
 - $f(z) = \max(0, z)$
 - Jednoduchá, rýchla, efektívna pre hlboké siete
 - Žiadny vanishing gradient pre kladné hodnoty
 - Problém: dying ReLU – neuróny s trvale záporným vstupom majú nulový gradient a „odumrú“
- Leaky ReLU
 - $f(z) = z$ ak $z > 0$, inak αz (napr. $\alpha = 0.01$)
 - Opravuje dying ReLU – záporné hodnoty majú malý, ale nenulový gradient.
- ELU (Exponential Linear Unit)
 - Hladká verzia Leaky ReLU, lepšia konvergencia, no pomalší výpočet.
- Softmax
 - $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$, výstup $\in (0, 1)$, súčet = 1
 - Používa sa výlučne vo výstupnej vrstve pri viactriednej klasifikácii.
 - Interpretácia: pravdepodobnostné rozdelenie cez triedy.
- Prehľad vhodnosti
 - Skryté vrstvy FNN/CNN: ReLU (alebo Leaky ReLU)
 - Výstup pri binárnej klasifikácii: sigmoid
 - Výstup pri viactriednej klasifikácii: softmax
 - Výstup pri regresii: lineárna (bez aktivácie)
 - RNN / rekurentné siete: tanh (centrovnosť pomáha stabilite)

Dopredný výpočet, stratová funkcia a chyba

- Dopredný výpočet (Forward pass)
 - Pre vrstvu l : $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
 - $\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$
 - Vstup $\mathbf{x}$ prechádza vrstvami až po výstup $\hat{\mathbf{y}}$.
- Maticový zápis pre dávku (batch)
 - Vstupná matica $\mathbf{X} \in \mathbb{R}^{N \times d}$ (N príkladov, d príznakov)
 - Pre vrstvu: $\mathbf{Z} = \mathbf{X} \mathbf{W}^T + \mathbf{b}$ (broadcasting biasu)
 - Umožňuje paralelné spracovanie celého batchu na GPU.
- Stratová funkcia (Loss function)
 - Meria, ako ďaleko je predikcia $\hat{\mathbf{y}}$ od skutočnej hodnoty $\mathbf{y}$.
 - Je to to, čo minimalizujeme počas tréningu.
- MSE (Mean Squared Error) – pre regresiu
 - $L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$

- Citlivá na odľahlé hodnoty (outliers) – kvadratický trest.
- MAE (Mean Absolute Error) – pre regresiu
 - $L = \frac{1}{N} \sum_i |\hat{y}_i - y_i|$
 - Robustnejšia voči outlierom.
- Binary Cross-Entropy – binárna klasifikácia
 - $L = -\frac{1}{N} \sum_i [y_i \log \hat{y}_i + (1-y_i) \log (1-\hat{y}_i)]$
 - Výstupná vrstva: sigmoid.
- Categorical Cross-Entropy – viactriedna klasifikácia
 - $L = -\frac{1}{N} \sum_i \sum_c y_{i,c} \log \hat{y}_{i,c}$
 - Výstupná vrstva: softmax.
- Chyba (metric) vs. stratová funkcia
 - Stratová funkcia: diferencovateľná funkcia, ktorú backprop minimalizuje (napr. cross-entropy).
 - Chyba / metrika: zrozumiteľný ukazovateľ výkonu, nemusí byť diferencovateľný (napr. accuracy, F1).
 - Optimalizujeme stratovú funkciu $\rightarrow$ dúfame, že metrika sa zlepší.

Backpropagation a úprava váh

- Princíp
 - Cieľ: nájsť váhy W , ktoré minimalizujú stratu L .
 - Metóda: gradient descent – pohybujeme sa v smere záporného gradientu straty.
 - Gradient descent (GD)
 - $W \leftarrow W - \eta \frac{\partial L}{\partial W}$
 - η (learning rate) – veľkosť kroku.
 - Backpropagation – algoritmus
- Forward pass: vypočítame $\hat{y}$ a stratu L .**
 - Spätný prechod: pomocou reťazového pravidla (chain rule) vypočítame $\frac{\partial L}{\partial W^{(l)}}$ pre každú vrstvu sprava doľava.**
 - Aktualizácia váh: $W^{(l)} \leftarrow W^{(l)} - \eta \nabla_{W^{(l)}} L$**
- Chain rule (reťazové pravidlo)
 - $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$
 - Gradient prechádza spätne vrstvami – každá vrstva preposiela gradient ďalej dozadu.
 - Vanishing gradient
 - Pri hlbokých sieťach s sigmoid/tanh sa gradient môže znásobiť malými hodnotami a „vymiznúť“ – váhy vo včasných vrstvách sa takmer neaktualizujú.
 - Riešenie: ReLU, batch normalization, reziduálne spojenia.
 - Exploding gradient
 - Gradient rastie exponenciálne – nestabilný tréning.

- Riešenie: gradient clipping (orezanie gradientu na max. hodnotu).

Optimalizátory

- Úloha optimalizátora
 - Určuje, ako sa aktualizujú váhy na základe gradientu.
 - Stratová funkcia hovorí „čo minimalizovať“, optimalizátor hovorí „ako kráčať“.
- SGD (Stochastic Gradient Descent)
 - Aktualizácia na základe jedného príkladu alebo mini-batchu.
 - Jednoduchý, ale pomalá konvergencia, citlivý na learning rate.
- SGD s momentumom
 - Pridáva váhu predchádzajúcim aktualizáciám: $v \leftarrow \beta v + \nabla L$, $W \leftarrow W - \eta v$
 - Rýchlejšia konvergencia, menej oscilácie.
- RMSprop
 - Adaptívna learning rate pre každý parameter: delí gradient štandardnou odchýlkou nedávnych gradientov.
 - Vhodné pre nestacionárne problémy a RNN.
- Adam (Adaptive Moment Estimation)
 - Kombinuje momentum (1. moment – priemer gradientov) a RMSprop (2. moment – rozptyl gradientov).
 - $m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L$ (moment)
 - $v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla L)^2$ (rozptyl)
 - $W \leftarrow W - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$
 - Predvolené hodnoty: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$
 - Najpoužívanější optimalizátor v praxi – robustný, nevyžaduje ručné nastavenie LR.
- Porovnanie
 - SGD: jednoduchý, ale potrebuje starostlivé nastavenie LR a momenta.
 - Adam: rýchla konvergencia, vhodný pre väčšinu problémov.
 - V niektorých prípadoch SGD s momentumom generalizuje lepšie ako Adam (napr. v počítačovom videní).

Typy neurónových sietí

- FNN – Feedforward Neural Network (dopredná sieť)
 - Informácia tečie jedným smerom: vstup → skryté vrstvy → výstup.
 - Bez cyklov a spätných väzieb.
 - Vhodná pre tabuľkové dáta, jednoduchú klasifikáciu/regresiu.
- CNN – Convolutional Neural Network
 - Špeciálne vrstvy: konvolučné filtre, pooling.

- Lokálne zdieľanie váh – efektívne pre priestorové vzory.
- Vhodná pre obrázky, video, 1D signály.
- RNN – Recurrent Neural Network
 - Obsahuje spätnú slučku – výstup vrstvy je vstupom v ďalšom časovom kroku.
 - Pamätá kontext z predchádzajúcich krokov.
 - Vhodná pre sekvenčné dáta (text, časové rady, reč).
 - Problémy: vanishing gradient pri dlhých sekvenciách.
- LSTM (Long Short-Term Memory)
 - Špeciálny typ RNN s bránami (forget, input, output gate).
 - Lepšie zachytáva dlhodobé závislosti ako štandardná RNN.
- GRU (Gated Recurrent Unit)
 - Zjednodušená verzia LSTM – menej parametrov, podobná výkonnosť.
- Autoencoder
 - Encoder stlačí vstup do kompaktnej reprezentácie (latent space), Decoder ho rekonštruje.
 - Využitie: redukcia dimenzionality, detekcia anomálií, denoising.
- GAN (Generative Adversarial Network)
 - Generátor vs. diskriminátor – súťažia navzájom.
 - Generátor produkuje syntetické dáta, diskriminátor rozlišuje reálne/falošné.
- Transformer
 - Mechanizmus attention (self-attention).
 - Základ moderných jazykových modelov (BERT, GPT).
 - Nemá rekurentnosť – paralelizovateľný tréning.

Vplyv hyperparametrov na tréovanie

- Čo sú hyperparametre?
 - Parametre, ktoré nenastavuje backpropagation, ale nastavuje ich dizajnér.
 - Nastavenie ovplyvňuje výsledky, rýchlosť konvergenzie a generalizáciu.
- Hlavné hyperparametre
 - Learning rate η – najdôležitejší, viz. vlastná podtéma.
 - Veľkosť batchu – viz. vlastná podtéma.
 - Počet epoch – viz. vlastná podtéma.
 - Architektúra siete – počet vrstiev, počet neurónov, typ aktivačnej funkcie.
 - Regularizácia – dropout rate, L1/L2.
 - Optimalizátor – SGD, Adam, RMSprop, hodnoty β_1 , β_2 .
 - Inicializácia váh – Xavier, He.
- Vzťahy medzi hyperparametrami
 - Väčší batch $\rightarrow$ možno zvýšiť LR (linear scaling rule).
 - Hlbšia sieť $\rightarrow$ potrebuje starostlivejšiu inicializáciu a normalizáciu.

- Viac parametrov → vyššie riziko overfitting → väčší dropout.

Epocha, batch, iterácia – vzťahy a rozmery matíc

- Epocha
 - Jedno úplné prechod celým tréovacím datasetom.
 - Po jednej epoche sieť videla každý tréovací príklad raz.
- Batch (mini-batch)
 - Podmnožina tréovacích príkladov spracovaná naraz.
 - Na základe jedného batchu sa vypočíta gradient a aktualizujú váhy.
 - Typické veľkosti: 16, 32, 64, 128, 256.
- Iterácia
 - Jeden krok aktualizácie váh = spracovanie jedného batchu.
 - $\text{iterácií za epochu} = \lceil N / B \rceil$, kde N = počet príkladov, B = veľkosť batchu.
 - Príklad: 10 000 príkladov, batch 100 → 100 iterácií na epochu.
- Rozmer matíc
 - Batch vstupov: $X \in \mathbb{R}^{B \times d}$ (B príkladov, d príznakov)
 - Váhová matica vrstvy: $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$
 - Výstup vrstvy: $Z = XW^T + \mathbf{b} \in \mathbb{R}^{B \times d_{\text{out}}}$
 - Väčší batch → väčšia matica → viac pamäte GPU, ale efektívnejší výpočet.
- SGD vs. mini-batch vs. full-batch GD
 - SGD ($B=1$): veľký šum, pomalé, väčšia šanca uniknúť lokálnemu minimu.
 - Mini-batch: kompromis – paralelné GPU výpočty + primeraný šum.
 - Full-batch ($B=N$): presný gradient, ale pomalé a pamäťovo náročné.

Fenomén preučenia a podučenia

- Underfitting (podučenie)
 - Model je príliš jednoduchý – nedokáže zachytiť vzory v dátach.
 - Znaky: vysoká chyba na tréovacej aj validačnej množine.
 - Príčiny: príliš malá sieť, príliš málo epoch, príliš silná regularizácia.
 - Riešenie: väčšia sieť, viac epoch, menej regularizácie.
- Overfitting (preučenie)
 - Model sa príliš prispôsobí tréovacím dátam – pamätá si šum a konkrétne príklady.
 - Znaky: nízka chyba na tréovaní, vysoká chyba na validácii/teste.
 - Príčiny: príliš zložitá sieť, príliš veľa epoch, málo dát.
 - Riešenie: dropout, regularizácia, early stopping, augmentácia, viac dát.
- Cieľ: dobrá generalizácia
 - Model musí fungovať na nových, nevidených dátach.

- Sledujeme validačnú metriku – nie len tréningovú.

Inicializácia váh pred tréňovaním

- Prečo záleží?
 - Zlá inicializácia → vanishing alebo exploding gradient hneď na začiatku.
 - Nulová inicializácia: všetky neuróny počítajú to isté → problém symetrie, gradientné aktualizácie sú identické → sieť sa nenaučí nič užitočné.
- Náhodná malá inicializácia
 - Váhy z $\mathcal{N}(0, \sigma^2)$ s malým σ .
 - Jednoduché, ale pri hĺbkych sieťach stále nestabilné.
- Xavier / Glorot inicializácia
 - $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$
 - Navrhnuté pre tanh a sigmoid aktivácie.
 - Zachováva rozptyl aktivácií a gradientov v celej sieti.
- He inicializácia
 - $\sigma^2 = \frac{2}{n_{\text{in}}}$
 - Navrhnuté špeciálne pre ReLU aktivácie.
 - Najpoužívanějšía inicializácia pri moderných sieťach s ReLU.
- Praktické odporúčanie
 - ReLU + He inicializácia: štandardná kombinácia.
 - Tanh/Sigmoid + Xavier: vhodnejšie pre tieto aktivácie.

Monitorovanie tréningu

- Čo sledujeme
 - Train loss: klesá → sieť sa učí.
 - Validation loss (val_loss): klesá → generalizácia sa zlepšuje; rastie → preučenie.
 - Train accuracy / val accuracy: presnosť na tréningových / validačných dátach.
- Tréningová vs. validačná metrika
 - Sledujeme obe – samotná tréningová nestačí.
 - Ak train ↓, val ↑ → preučenie.
 - Ak train ↑, val ↑ (obe zlé) → podučenie.
 - Cieľ: train ≈ val a obe nízke (strata) / vysoké (presnosť).
- Znaky preučenia v grafoch
 - Val loss začne rásť, zatiaľ čo train loss klesá.
 - Nožnicový tvar kriviek.
- Train / Validation / Test rozdelenie

- Train: na aktualizáciu váh.
 - Validation: na monitorovanie a nastavenie hyperparametrov.
 - Test: záverečné hodnotenie – používa sa len raz, žiadne rozhodnutia podľa neho počas vývoja.
 - Frekvencia monitorovania
 - Zvyčajne po každej epoche.
 - Pri veľkých datasetoch aj po každých N iteráciách.
-

Ukončenie tréningu – Early Stopping a limit epoch

- Limit počtu epoch
 - Najjednoduchšia stratégia – zastavíme po vopred danom počte epoch.
 - Riziko: zastavíme príliš skoro (underfitting) alebo príliš neskoro (overfitting).
 - Early Stopping
 - Monitorujeme val_loss (alebo inú metriku).
 - Ak sa výrazne nezlepší za posledných patience epoch → zastavíme tréning.
 - Patience: koľko epoch čakáme na zlepšenie (napr. 10–20).
 - Typicky sa uloží model s najlepším val_loss (checkpoint) a po zastavení sa obnoví.
 - Výhody early stopping
 - Automaticky zabráni preučeniu.
 - Šetrí čas – nepokračujeme, ak sa nič nezlepšuje.
 - Evidencia behov (experiment tracking)
 - Zaznamenávanie hyperparametrov, metrík a modelov pre každý beh.
 - Nástroje: MLflow, Weights & Biases, TensorBoard.
 - Umožňuje porovnávať experimenty a reprodukovat' výsledky.
-

Vplyv learning rate na priebeh učenia

- Príliš vysoké LR
 - Veľké kroky → gradient „preskakuje“ cez minimum → osciluje alebo diverguje.
 - Loss rastie alebo sa nevedie ku konvergencii.
 - Graf loss: chaotický, nestabilný.
- Príliš nízke LR
 - Veľmi malé kroky → pomalá konvergencia.
 - Model sa môže zaseknúť v lokálnom minime alebo konvergovať veľmi dlho.
 - Graf loss: klesá veľmi pomaly.
- Vhodné LR
 - Graf loss: plynulo klesá a stabilizuje sa.
 - Typické hodnoty: 10^{-3} pre Adam, 10^{-1} – 10^{-2} pre SGD (závisí od problému).

- LR scheduling (plánovanie LR)
 - Step decay: zníženie LR po X epochách.
 - Exponential decay: postupné znižovanie.
 - Cosine annealing: LR klesá podľa kosínusovej krivky.
 - Warm-up: začíname s malým LR, postupne zvyšujeme → stabilizácia na začiatku tréningu.
 - Learning Rate Finder
 - Technika: postupne zvyšujeme LR a sledujeme loss → vyberieme LR tesne pred explóziou.
-

Vplyv veľkosti batchu

- Malý batch (napr. 16–32)
 - Viac šumu v gradiente → nepravidelná trajektória v priestore parametrov.
 - Šum môže pomôcť uniknúť lokálnym minimám → tendenciu k lepšej generalizácii.
 - Viac iterácií na epochu → pomalší výpočet na GPU (slabé využitie paralelizmu).
 - Vyžaduje menej pamäte.
 - Veľký batch (napr. 512–4096)
 - Presnejší odhad gradientu → stabilnejší tréning.
 - Lepšie využitie GPU paralelizmu → rýchlejšia epocha.
 - Tendencia konvergovať k ostrým minimám → horšia generalizácia (tzv. „sharp minima“ problém).
 - Vyžaduje viac GPU pamäte.
 - Praktické odporúčanie
 - Začni s batch 32 alebo 64 a experimentuj.
 - Pri zväčšení batchu zvyčajne treba zodpovedajúco zvýšiť LR (linear scaling rule).
 - Obmedzenia: GPU pamäť obmedzuje maximálnu veľkosť batchu.
-

Voľba počtu epoch

- Príliš málo epoch → underfitting, model sa nenastačil naučiť.
 - Príliš veľa epoch → overfitting (bez early stopping alebo regularizácie).
 - Správna voľba
 - Monitorovanie val_loss + early stopping = automaticky.
 - Ak nie je early stopping: sledovanie grafov, zastavenie pri začiatku rastu val_loss.
 - Orientačne: malé datasety a modely – desiatky epoch; veľké datasety a CNN – stovky epoch.
 - Transfer learning
 - Fine-tuning predtrénovaného modelu: stačí oveľa menej epoch (5–30).
 - Tréning od nuly: potrebuje viac epoch.
-

Datsety – train/val/test split a benchmark datasety

- Prečo deliť dataset?
 - Testujeme generalizáciu na dátach, ktoré model počas tréningu nevidel.
 - Validačná množina: ladenie hyperparametrov (model ich „nepriamo vidí“).
 - Testovacia množina: záverečné, nezaujaté hodnotenie.
- Typické pomery
 - 80 % train / 10 % val / 10 % test.
 - 70 % / 15 % / 15 %.
 - Pri malom datasete: väčší podiel val/test alebo cross-validation.
- Zásady
 - Testovacia množina sa nikdy nepoužíva na rozhodovanie počas vývoja.
 - Train a val možno náhodne premiešať, test musí zostať fixný.
 - Pri časových radoch: nemiešame – test je vždy z neskoršieho obdobia (data leakage).
- Benchmark datasety
 - MNIST: ručne písané číslice, 70 000 obrázkov 28×28 – jednoduchý benchmark.
 - CIFAR-10 / CIFAR-100: farebné obrázky 32×32, 10 alebo 100 tried.
 - ImageNet: 1,2 M obrázkov, 1000 tried – základ pre predtrénované CNN.
 - COCO: detekcia objektov a segmentácia.
 - Benchmarky umožňujú porovnanie modelov medzi výskumníkmi.

Dropout a augmentácia v kontexte preučenia

- Dropout
 - Počas tréningu náhodne deaktivuje (nastaví na 0) určité percento neurónov v každom forward passe.
 - Hyperparameter: `dropout_rate` (napr. 0.2–0.5).
 - Efekt: sieť sa nemôže spoliehať na konkrétne neuróny → musí naučiť redundanntné reprezentácie → lepšia generalizácia.
 - Počas inferencie (test/predikcia): dropout je vypnutý, váhy sa škálujú.
 - Typicky sa pridáva za plne prepojené vrstvy; v CNN menej (niekedy za posledné konvolučné bloky).
- L1 / L2 regularizácia (weight decay)
 - Pridáva penaltu za veľkosť váh do stratovej funkcie.
 - L2 (Ridge): $L_{\text{reg}} = L + \lambda \sum w^2$ → tlačí váhy k nule, ale nie na nulu.
 - L1 (Lasso): $L_{\text{reg}} = L + \lambda \sum |w|$ → môže tlačiť váhy presne na nulu (sparstá riešenia).
- Augmentácia obrázkov
 - Umelé rozšírenie trénovacieho datasetu transformáciami vstupov:
 - Otočenie, horizontálne/vertikálne prevrátenie.
 - Náhodný rez (random crop), zmena jasú/kontrastu.

- Gaussovský šum, priemerné rozmazanie.
- Sieť vidí „nové“ príklady → ťažšie sa preučí na konkrétne obrázky.
- Augmentácia sa aplikuje len na trénovaciu množinu, nie na val/test.

Cross-validation a shuffling dát

- K-fold cross-validation
 - Dataset sa rozdelí na k rovnakých dielov (foldov).
 - Tréning sa opakuje k-krát: vždy iný fold je validačný, zvyšok trénovací.
 - Výsledná metrika = priemer cez k behov.
 - Výhoda: lepší odhad generalizácie, každý príklad bol raz validačný.
 - Nevýhoda: k-násobne dlhší tréning.
- Stratified K-fold
 - Zachováva proporcie tried v každom folde → dôležité pri nevyvážených datasetoch.
- Náhodný výber (shuffling)
 - Pred rozdelením na train/val náhodne zamiešame dáta → zabránime systematickým chybám.
 - Ak je dataset zoradený podľa triedy, bez shufflingu by train/val neobsahoval všetky triedy.
- Prečo je test vždy fixný?
 - Ak by sme test menili alebo sa na neho pozreli pri ladení, stáva sa de facto validačnou množinou.
 - Výsledok by bol optimisticky skreslený – model „vidí“ testové dáta.
 - Fixný test = garanciou nezaujatého záverečného hodnotenia.
 - Pri cross-validation: outer test set zostáva vždy mimo, inner loop robí train/val split.

Hľadanie hyperparametrov

- Prečo potrebujeme systematické hľadanie?
 - Ručné ladenie je pomalé a subjektívne.
 - Hyperparametre sú vzájomne závislé (napr. LR a batch size).
- Grid Search
 - Vyskúšame všetky kombinácie z vopred definovanej mriežky hodnôt.
 - Jednoduchý, ale exponenciálne drahý s počtom hyperparametrov.
- Random Search
 - Náhodne vzorkujeme kombinácie hyperparametrov.
 - Efektívnejší ako grid search pri mnoho hyperparametroch (niektoré hyperparametre nemajú veľký vplyv).
- Genetický algoritmus (GA)
 - Populácia konfigurácií hyperparametrov, kríženie a mutácia → evolvujú lepšie konfigurácie.
 - Vhodné pre veľký priestor hľadania.
- Optuna / Bayesian Optimization

- Stavíme pravdepodobnostný model (surrogate model) výkonu na základe doterajších pokusov.
 - Nové pokusy vyberáme tam, kde je pravdepodobnosť zlepšenia najvyššia.
 - Efektívnejší ako random search – rýchlejšie konverguje k dobrým hodnotám.
 - Optuna: moderná knižnica pre Bayesianovu optimalizáciu hyperparametrov.
 - Praktické odporúčanie
 - Málo hyperparametrov (1–3): grid search.
 - Viac hyperparametrov: random search alebo Optuna.
 - Veľmi drahý tréning: Optuna / Bayesian.
-

Generalizácia verzus učenie naspamäť

- Generalizácia
 - Schopnosť modelu správne odpovedať na nové, nevidené príklady.
 - Cieľ každého ML modelu.
 - Meranie: výkon na testovacej množine.
 - Učenie naspamäť (memorization)
 - Model si zapamätá trénovacie príklady namiesto naučenia sa všeobecných vzorov.
 - Extrémny overfitting.
 - Symptóm: takmer nulová chyba na tréningu, vysoká chyba na teste.
 - Čo znamená „zvládnuť nové príklady“
 - Model extrahuje abstraktné reprezentácie a pravidlá (nie konkrétne príklady).
 - Príklad: rozpoznávanie mačiek → model sa naučí rysy (uši, srst', oči), nie konkrétne fotografie.
 - Šum v dátach
 - Zašumené vstupy (nesprávne anotácie, šum v signáli) sťažujú učenie.
 - Model sa môže naučiť šum ako vzor → zhoršená generalizácia.
 - Riešenie: čistenie dát, augmentácia, robustné stratové funkcie.
-

Bias–Variance Trade-off

- Bias (skreslenie)
 - Chyba spôsobená príliš zjednodušenými predpokladmi modelu.
 - Vysoký bias → underfitting – model nepachytí skutočné vzory.
 - Príklad: lineárny model na nelineárnych dátach.
- Variance (rozptyl)
 - Citlivosť modelu na malé zmeny v trénovacích dátach.
 - Vysoká variance → overfitting – model sa príliš prispôsobí trénovacej množine.
 - Príklad: hlboká sieť s malým datasetom.
- Trade-off
 - Znižovanie biasu (zložitejší model) → zvyšovanie variance.

- Znižovanie variance (jednoduchší model / regularizácia) → zvyšovanie biasu.
 - Cieľ: nájsť rovnováhu s minimálnou celkovou chybou.
 - Vzťah k underfitting/overfitting
 - Underfitting = vysoký bias, nízka variance.
 - Overfitting = nízky bias, vysoká variance.
 - Dobrý model = nízky bias, nízka variance.
 - Diagnostika
 - Ak train $\approx$ val a obe zlé → vysoký bias → zložitejší model.
 - Ak train dobrý, val zlý → vysoká variance → regularizácia, viac dát, dropout.
-

Kapacita modelu

- Definícia
 - Schopnosť modelu aproximovať komplexné funkcie.
 - Závisí od počtu parametrov, hĺbky siete, šírky vrstiev, typu architektúry.
 - Príliš nízka kapacita
 - Model nevie zachytiť skutočné vzory → underfitting, vysoký bias.
 - Príliš vysoká kapacita
 - Model má veľkú tendenciu k overfittingu – pamätá si tréningové príklady.
 - Riziko rastie pri malom datasete.
 - Kedy pridať viac dát vs. zjednodušiť model?
 - Vysoká variance (overfitting): primárne riešenie – viac dát alebo regularizácia.
 - Vysoký bias (underfitting): primárne riešenie – väčší model, viac vrstiev, viac neurónov.
 - Viac dát málokedy zhorší situáciu, ale môže byť drahé alebo nedostupné.
 - Regularizácia ako kontrola kapacity
 - Dropout, weight decay (L2), batch normalization efektívne znižujú efektívnu kapacitu modelu.
-

Rôznorodosť testovacích príkladov a dát

- Prečo je rôznorodosť dôležitá?
 - Model trénovaný na úzkom spektre dát zlyháva pri iných podmienkach (iné osvetlenie, iní jedinci, iné zariadenie).
 - Testovacia množina musí pokrývať reálnu variabilitu vstupov.
- Distribučný posun (distribution shift)
 - Tréningové a testovacie dáta pochádzajú z rôznych distribúcií → výkon klesá.
 - Príklady: sezónne zmeny, iná kamera, iná demografia pacientov.
- Dôsledky nedostatočnej rôznorodosti
 - Model sa môže naučiť iba „ľahké“ prípady a zlyhávať na okrajových.

- Metriky na trénovacej množine môžu byť falošne optimistické.
 - Ako zabezpečiť rôznorodosť
 - Zbieranie dát z rôznych zdrojov, podmienok, zariadení.
 - Stratifikácia pri splitovaní – zabezpečenie zastúpenia všetkých podskupín.
 - Augmentácia – umelé zavádzanie variability.
 - Testovanie na „ťažkých" príkladoch
 - Okrem priemerného výkonu sledovať výkon na najhorších prípadoch (worst-case), minoritných triedach, hraničných príkladoch.
-